

Food Ordering System

💡 Problem Statement:

To design and implement a Food Ordering System using Python that allows users to select food items from different categories (like pizza, drinks, burgers, desserts), add items with desired quantities to the cart, apply discount coupons, calculate total payable amount with tax (GST), preview the order summary, and finally save the bill as a text file.

The system should provide an attractive user interface using Tkinter, along with additional features such as dice animation, sound toggle, and real-time total calculation.

📁 Concepts Used:

Concept	Purpose
Tkinter (UI)	For designing the interactive GUI (buttons, labels, dropdowns, billing area)
Functions	For modular operations like adding items, applying coupons, calculating totals, saving bills
Dictionaries	To store menu items and prices under categories
Lists	To maintain the dynamic cart with multiple food items
String Formatting	For displaying formatted bills and summaries
File Handling	Saving the final bill into a text file
Modules	Use of pre-built modules (Tkinter, datetime, winsound)
Conditional Statements	Validating coupons, checking empty cart, verifying quantity
Loops	Iterating over cart items to display or compute totals

📄 Short Description of Working:

1. The system loads multiple food categories such as Pizzas, Burgers, Drinks, and Desserts.
2. The user selects an item and quantity using dropdown menus and input boxes.
3. The selected item is added to the cart, where it can accumulate multiple entries.
4. The order total is updated automatically, including:
 - ✓ GST of 12%
 - ✓ Discount coupons (e.g., SAVE50, NEW10, FEST20)
5. The user may apply or remove a coupon anytime before checkout.
6. A preview of the final bill is shown with all details (item name, qty, item total, tax, discount, grand total).

7. On confirming, the bill is saved into a text file using an auto-increment bill number (e.g., Order_Bill_No_003.txt).
8. The system provides options for:
 - Restarting the order
 - Sound ON/OFF control for interactive ordering
 - User-friendly, colorful UI with real-time updates

◆ **Logic Flow:**

1. Start
2. Load categories, price list, and initialize cart.
3. Wait for user actions (Add Item, Apply Coupon, Toggle Sound).
4. When an item is selected → store item & quantity into cart.
5. Update subtotal, discount, GST, and grand total.
6. If coupon button clicked → validate and update discount.
7. On checkout:
 - Generate bill
 - Save to text file
 - Display confirmation
8. End / Restart.

 foodOrderingSystem.py

```

1  # FOOD ORDERING SYSTEM (CONSOLE PROJECT)
2
3  import datetime
4
5  # ----- Configuration -----
6
7  TAX_RATE = 0.12 # 12% GST
8
9  MENU = {
10     "Pizzas": {
11         1: ("Margherita", 250),
12         2: ("Farmhouse", 320),
13         3: ("Pepperoni", 350),
14     },
15     "Burgers": {
16         4: ("Veg Burger", 150),
17         5: ("Cheese Burger", 180),
18         6: ("Chicken Burger", 220),
19     },
20     "Drinks": {
21         7: ("Coke", 60),
22         8: ("Lemonade", 80),
23         9: ("Cold Coffee", 120),
24     },
25     "Desserts": {
26         10: ("Brownie", 140),
27         11: ("Ice Cream", 90),
28         12: ("Cheesecake", 180),
29     }
30 }
31
32 COUPONS = {
33     "SAVE50": {"type": "flat", "value": 50}, # flat ₹50 off
34     "NEW10": {"type": "percent", "value": 10}, # 10% off
35     "FEST20": {"type": "percent", "value": 20} # 20% off
36 }
37
38 # ----- Helper Functions -----
39
40
41 def print_header(): # usage
42     print("\n" + "=" * 40)
43     print("      FOOD ORDERING SYSTEM      ")
44     print("=" * 40)
45
46
47 def display_menu(): # 2 usages
48     print("\n==== MENU =====")
49     for category, items in MENU.items():
50         print(f"\n-- {category} --")
51         for code, (name, price) in items.items():
52             print(f"{code}. {name} - ₹{price}")
53
54 def find_item_by_code(code): # 1 usage
55     """Return (name, price) given item code, else None."""
56     for items in MENU.values():
57         if code in items:
58             return items[code]
59     return None
60
61 def display_cart(cart): # 2 usages
62     if not cart:
63         print("\nYour cart is empty.")
64         return
65
66     print("\n==== YOUR CART =====")
67     print("{<4} {<20} {<8} {<8}".format(*args: "No.", "Item", "Qty", "Total"))
68     print("-" * 40)
69     for idx, item in enumerate(cart, start=1):
70         name = item["name"]
71         qty = item["quantity"]
72         price = item["price"]
73         line_total = qty * price
74         print(f"{idx:<4} {name:<20} {qty:<8} ₹{line_total:<8}")
75     print("-" * 40)

```

```

75     print("\n")
76
77     def calculate_subtotal(cart): 1 usage
78         return sum(item["price"] * item["quantity"] for item in cart)
79
80     def apply_coupon(subtotal, code): 1 usage
81         """Return (discount_amount, coupon_used)."""
82         if not code:
83             return 0, None
84
85         code = code.upper()
86         if code not in COUPONS:
87             print("Invalid coupon code. No discount applied.")
88             return 0, None
89
90         coupon = COUPONS[code]
91         if coupon["type"] == "flat":
92             discount = coupon["value"]
93         else: # percent
94             discount = (coupon["value"] / 100) * subtotal
95
96         # Avoid negative totals
97         discount = min(discount, subtotal)
98
99         print(f"Coupon '{code}' applied. Discount: ₹{int(discount)}")
100        return int(discount), code
101
102    def save_bill_to_file(cart, subtotal, taxAmount, discount, total, coupon_code): 1 usage
103        now = datetime.datetime.now()
104        filename = f"bill_{now.strftime('%Y%m%d_%H%M%S')}.txt"
105
106        with open(filename, "w", encoding="utf-8") as f:
107            f.write("==== FOOD ORDERING SYSTEM - BILL ====\n")
108            f.write(f>Date & Time: {now}\n")
109            f.write("-" * 40 + "\n")
110            f.write("{:<4} {:<20} {:<8} {:<8}\n".format(*args, "No.", "Item", "Qty", "Total"))
111            f.write("-" * 40 + "\n")
112
113            for idx, item in enumerate(cart, start=1):
114                name = item["name"]
115                qty = item["quantity"]
116                price = item["price"]
117                lineTotal = qty * price
118                f.write(f"{idx:<4} {name:<20} {qty:<8} ₹{lineTotal:<8}\n")
119
120            f.write("-" * 40 + "\n")
121            f.write(f"Subtotal: ₹{subtotal}\n")
122            f.write(f"GST @ {int(TAX_RATE * 100)}%: ₹{taxAmount}\n")
123            if coupon_code:
124                f.write(f"Coupon ({coupon_code}) Discount: -₹{discount}\n")
125            f.write(f"Grand Total: ₹{total}\n")
126            f.write("Thank you for ordering with us!\n")
127
128        print(f"\nBill saved as '{filename}'")
129
130
131    # ----- Main Flow -----
132
133    def main(): 1 usage
134        cart = []
135        print_header()
136
137        while True:
138            print("\n1. View Menu")
139            print("2. Add Item to Cart")
140            print("3. View Cart")
141            print("4. Checkout")
142            print("5. Exit")
143
144            choice = input("Enter your choice (1-5): ").strip()
145
146            if choice == "1":
147                display_menu()
148

```

```

149 elif choice == "2":
150     display_menu()
151     try:
152         code = int(input("\nEnter item code to add: "))
153         item = find_item_by_code(code)
154         if item is None:
155             print("Invalid item code.")
156             continue
157
158         name, price = item
159         qty = int(input(f"Enter quantity for {name}: "))
160         if qty <= 0:
161             print("Quantity must be positive.")
162             continue
163
164         # Check if item already in cart -> update quantity
165         for cItem in cart:
166             if cItem["name"] == name:
167                 cItem["quantity"] += qty
168                 break
169         else:
170             cart.append({"name": name, "price": price, "quantity": qty})
171
172         print(f"{qty} x {name} added to cart.")
173
174     except ValueError:
175         print("Please enter valid numeric values.")
176
177 elif choice == "3":
178     display_cart(cart)
179
180 elif choice == "4":
181     if not cart:
182         print("\nYour cart is empty. Add items before checkout.")
183         continue
184
185     # Show order summary
186     display_cart(cart)
187     subtotal = calculate_subtotal(cart)
188     tax_amount = int(subtotal * TAX_RATE)
189     print(f"\nSubtotal: ₹{subtotal}")
190     print(f"GST @ {int(TAX_RATE * 100)}%: ₹{tax_amount}")
191
192     # Apply coupon
193     couponInput = input("Enter coupon code (or press Enter to skip): ").strip()
194     discount, couponUsed = apply_coupon(subtotal, couponInput)
195
196     total = subtotal + tax_amount - discount
197     total = int(total)
198
199     print("\n==== ORDER SUMMARY =====")
200     print(f"Subtotal: ₹{subtotal}")
201     print(f"GST: ₹{tax_amount}")
202     if couponUsed:
203         print(f"Coupon ({couponUsed}) Discount: -₹{discount}")
204     print(f"Grand Total: ₹{total}")
205
206     confirm = input("\nConfirm order? (y/n): ").strip().lower()
207     if confirm == "y":
208         print("\nOrder confirmed! 🎉")
209         save_choice = input("Do you want to save the bill to a file? (y/n): ").strip().lower()
210         if save_choice == "y":
211             save_bill_to_file(cart, subtotal, tax_amount, discount, total, couponUsed)
212             print("Thank you for ordering. Visit again!")
213             break
214         else:
215             print("Order cancelled. You can continue modifying your cart.")
216
217     elif choice == "5":
218         print("\nExiting... Thank you!")
219         break
220
221     else:
222         print("Invalid choice. Please select between 1-5.")
223
224
225 # Run the program
226 if __name__ == "__main__":
227     main()
228

```

Output Screenshot (Representation):

```

↓
≡
≡
≡
1. View Menu
2. Add Item to Cart
3. View Cart
4. Checkout
5. Exit
Enter your choice (1-5): 1

=====
FOOD ORDERING SYSTEM
=====

===== MENU =====

-- Pizzas --
1. Margherita - ₹250
2. Farmhouse - ₹320
3. Pepperoni - ₹350

-- Burgers --
4. Veg Burger - ₹150
5. Cheese Burger - ₹180
6. Chicken Burger - ₹220

-- Drinks --
7. Coke - ₹60
8. Lemonade - ₹80
9. Cold Coffee - ₹120

-- Desserts --
10. Brownie - ₹140
11. Ice Cream - ₹90
12. Cheesecake - ₹180

1. View Menu
2. Add Item to Cart
3. View Cart
4. Checkout
5. Exit
Enter your choice (1-5): |

```

Food Ordering System

💡 Problem Statement:

To design and implement a **Personal Notebook / Reminder App** using Python that allows users to write and store daily notes or reminders with an automatic timestamp. The application should enable users to **add, view, search, edit, and delete notes** through a user-friendly interface. All notes must be stored permanently using **file handling**, and the system should update the saved data whenever the user modifies it.

📚 Concepts Used:

Concept	Purpose
File Handling	To permanently store notes into a text file and retrieve them on startup
Functions	To modularize features like add, search, update, delete, and save
Strings & Searching	Used to search keywords inside saved notes
Dictionaries	To store note data (timestamp + message) in structured form
Lists	To maintain multiple notes in memory
Date & Time (datetime)	To add automatic timestamps for every note
Tkinter GUI	For creating an interactive and user-friendly interface
Conditional Statements	For validating user actions such as empty notes, invalid selection, etc.
Looping Structures	To display and refresh multiple notes in the UI

📄 Short Description of Working:

1. The system loads existing notes from a text file at startup using file handling.
2. The user writes a note in the input area and clicks **Add Note**.
3. The note gets automatically stamped with the current **date & time** and is stored in the file.
4. Users can view all notes in a scrollable table view.
5. A **Search feature** allows finding notes using keywords (e.g., “meeting”, “assignment”, etc.)
6. Users may select any note to **edit** or **delete** using dedicated buttons.
7. When an update or delete action occurs, the system overwrites the text file with new data.
8. All functionalities refresh dynamically without restarting the program.

◆ **Logic Flow:**

1. **Start Application**
2. Load notes from notes.txt
3. Wait for user action:
 - **Add Note**
 - **Search**
 - **Edit**
 - **Delete**
 - **Show All**
4. According to user action:
 - Add → Append new note + timestamp → Save file
 - Search → Filter notes by keyword
 - Edit/Delete → Select note from list → Modify or remove → Save file
5. Refresh notes view in UI
6. Exit if user closes window

 remainderNotebook.py

```

1  # PERSONAL NOTEBOOK / REMINDER APP (Console Project)
2
3  import os
4  import datetime
5
6  NOTES_FILE = "notes.txt"
7
8
9  # ----- Data Handling Functions -----
10
11 def load_notes(): 1usage
12     """Load notes from file into a list of dicts: [{'date':..., 'message':...}, ...]"""
13     notes = []
14     if not os.path.exists(NOTES_FILE):
15         return notes
16
17     with open(NOTES_FILE, "r", encoding="utf-8") as f:
18         lines = [line.rstrip("\n") for line in f]
19
20     i = 0
21     while i < len(lines):
22         if lines[i].strip() == "----- NOTE -----":
23             # Expect: Date line, Message line, separator
24             dateLine = lines[i + 1] if i + 1 < len(lines) else ""
25             msgLine = lines[i + 2] if i + 2 < len(lines) else ""
26             # Move index to after block
27             i = i + 4 # skip separator line too
28
29             if dateLine.startswith("Date:") and msgLine.startswith("Message:"):
30                 dateText = dateLine.replace(_old: "Date:", _new: "", _count: 1).strip()
31                 messageText = msgLine.replace(_old: "Message:", _new: "", _count: 1).strip()
32                 notes.append({'date': dateText, "message": messageText})
33             else:
34                 i += 1
35
36     return notes
37
38
39 def save_notes(notes): 3usages
40     """Overwrite file with all notes in the block format."""
41     with open(NOTES_FILE, "w", encoding="utf-8") as f:
42         for note in notes:
43             f.write("----- NOTE ----- \n")
44             f.write(f"Date: {note['date']} \n")
45             f.write(f"Message: {note['message']} \n")
46             f.write("----- \n")
47
48
49 # ----- Core Features -----
50
51 def add_note(notes): 1usage
52     print("\n 📝 ADD NEW NOTE")
53     text = input("Enter your note/reminder: ").strip()
54     if not text:
55         print("Note cannot be empty.")
56         return
57
58     now = datetime.datetime.now().strftime("%Y-%m-%d %H:%M")
59     notes.append({'date': now, "message": text})
60     save_notes(notes)
61     print("✅ Note added successfully with timestamp:", now)
62
63
64 def view_notes(notes): 1usage
65     print("\n 📖 ALL NOTES")
66     if not notes:
67         print("No notes found yet. Start by adding one!")
68         return
69
70     for idx, note in enumerate(notes, start=1):
71         print("-" * 40)
72         print(f"#{idx} 📅 {note['date']}")
73         print(f"📄 {note['message']}")
74         print("-" * 40)

```

```

75
76
77 def search_notes(notes): 3 usages
78     print("\n🔍 SEARCH NOTES")
79     keyword = input("Enter keyword to search: ").strip().lower()
80     if not keyword:
81         print("Keyword cannot be empty.")
82         return []
83
84     results = []
85     for i, note in enumerate(notes):
86         if keyword in note["message"].lower() or keyword in note["date"].lower():
87             results.append((i, note))
88
89     if not results:
90         print("No notes matched your search.")
91     else:
92         print(f"\nFound {len(results)} matching note(s):")
93         for idx, (original_index, note) in enumerate(results, start=1):
94             print("-" * 40)
95             print(f"Result {idx} (Note #{original_index + 1})")
96             print(f"📅 {note['date']}")
97             print(f"📄 {note['message']}")
98             print("-" * 40)
99
100     return results
101
102
103 def choose_from_search(results): 2 usages
104     """Let user pick one note from search results. Returns original index or None."""
105     if not results:
106         return None
107
108     try:
109         choice = int(input("Enter result number to select (0 to cancel): "))
110     except ValueError:
111         print("Invalid input.")
112         return None
113
114     if choice == 0:
115         return None
116
117     if 1 <= choice <= len(results):
118         original_index, _ = results[choice - 1]
119         return original_index
120
121     else:
122         print("Invalid choice.")
123         return None
124
125
126 def update_note(notes): 1 usage
127     print("\n✎ UPDATE NOTE (search-based)")
128     results = search_notes(notes)
129     index = choose_from_search(results)
130     if index is None:
131         return
132
133     old_note = notes[index]
134     print("\nCurrent note:")
135     print(f"📅 {old_note['date']}")
136     print(f"📄 {old_note['message']}")
137
138     new_text = input("\nEnter updated note text (leave blank to cancel): ").strip()
139     if not new_text:
140         print("Update cancelled.")
141         return
142
143     # Option: keep original date or update date. We'll update timestamp to show last edit.
144     now = datetime.datetime.now().strftime("%Y-%m-%d %H:%M")
145     notes[index] = {"date": now, "message": new_text}
146     save_notes(notes)
147     print("✅ Note updated successfully.")

```

```

148
149 def delete_note(notes): 1 usage
150     print("\n# DELETE NOTE (search-based)")
151     results = search_notes(notes)
152     index = choose_from_search(results)
153     if index is None:
154         return
155
156     note = notes[index]
157     print("\nYou are about to delete this note:")
158     print(f"📅 {note['date']}")
159     print(f"📄 {note['message']}")
160
161     confirm = input("Are you sure? (y/n): ").strip().lower()
162     if confirm == "y":
163         notes.pop(index)
164         save_notes(notes)
165         print("✅ Note deleted.")
166     else:
167         print("Deletion cancelled.")
168
169
170 # ----- Menu / Main Loop -----
171
172 def print_menu(): 1 usage
173     print("\n===== PERSONAL NOTEBOOK =====")
174     print("1. 📝 Add New Note")
175     print("2. 📖 View All Notes")
176     print("3. 🔍 Search Notes")
177     print("4. ✏️ Update Note (via search)")
178     print("5. 🗑️ Delete Note (via search)")
179     print("6. 🚪 Exit")
180     print("=====")
181
182
183 def main(): 1 usage
184     notes = load_notes()
185     print("👋 Welcome to Personal Notebook / Reminder App")
186     print(f"Using file: {NOTES_FILE}")
187
188     while True:
189         print_menu()
190         choice = input("Enter your choice (1-6): ").strip()
191
192         if choice == "1":
193             add_note(notes)
194         elif choice == "2":
195             view_notes(notes)
196         elif choice == "3":
197             search_notes(notes)
198         elif choice == "4":
199             update_note(notes)
200         elif choice == "5":
201             delete_note(notes)
202         elif choice == "6":
203             print("👋 Exiting Personal Notebook. Goodbye!")
204             break
205         else:
206             print("Invalid choice. Please select between 1 and 6.")
207
208
209 ▶ if __name__ == "__main__":
210     main()
211

```

Output Screenshot (Representation):

```

Welcome to Personal Notebook / Reminder App
Using file: notes.txt

===== PERSONAL NOTEBOOK =====
1. 📝 Add New Note
2. 📅 View All Notes
3. 🔍 Search Notes
4. ✏️ Update Note (via search)
5. 🗑️ Delete Note (via search)
6. 🚪 Exit
=====
Enter your choice (1-6): 1

📝 ADD NEW NOTE
Enter your note/reminder: Buy Fruits at Evening Walk
✅ Note added successfully with timestamp: 2025-12-06 16:27

===== PERSONAL NOTEBOOK =====
1. 📝 Add New Note
2. 📅 View All Notes
3. 🔍 Search Notes
4. ✏️ Update Note (via search)
5. 🗑️ Delete Note (via search)
6. 🚪 Exit
=====
Enter your choice (1-6): 2

📅 ALL NOTES
-----
#1 📅 2025-12-06 15:16
  ☐ Buy Fruits
-----
#2 📅 2025-12-06 16:27
  ☐ Buy Fruits at Evening Walk
-----

```